

Lab 4: NBA Player Evaluation Using Ridge Regression and the Lasso

Umais Siddiqui

2026-03-01

Table of contents

Overview	1
1 Problem 1: Ridge Regression for Inference	2
1.1 (a) Ordinary Linear Regression	2
1.2 (b) Examining RAPM Coefficients	4
1.3 (c) Ridge Regression RAPM	6
2 Problem 2: Possession Weights	9
2.1 (a) Heteroscedasticity in Stints	9
2.2 (b) Implementing Weighted Regression	11
3 Problem 3: Interpreting Bootstrap Uncertainty	16
3.1 (a) Bootstrap Confidence Intervals	16
3.2 (b) Impact of Minutes Played on Confidence Intervals	20
3.3 (c) Comparison to Bayesian Credible Intervals	21
4 Extra Credit: Ridge vs Lasso	25
4.1 Ridge vs Lasso RAPM Comparison	25

Overview

In this lab we use resampling techniques (cross-validation and the bootstrap) along with regularization (ridge regression and the lasso) to rank basketball players based on their performance during the 2022 NBA season. The central idea is **Regularized Adjusted Plus-Minus (RAPM)**: we model each stint (a continuous stretch where lineups remain fixed) as a linear function of which players are on the court, and recover a coefficient per player that measures their contribution to the team's scoring margin per 100 possessions.

```

nba_stints = pd.read_csv("DataSets/nba_stint_data.csv")

player_ids = pd.read_csv("DataSets/player_id.csv")

print(f"Stint data shape: {nba_stints.shape}")
print(f"Number of players: {player_ids.shape[0]}")
nba_stints.head(5)

```

Stint data shape: (32358, 546)

Number of players: 539

	game_id	stint_id	n_pos	home_points	away_points	minutes	margin	201939	202691	203110
0	22200002	1	14	5	2	2.700	21.429	1	1	1
1	22200002	2	9	6	2	1.670	44.444	1	1	1
2	22200002	3	5	0	3	0.480	-60.000	1	0	1
3	22200002	4	5	5	1	0.780	80.000	1	0	1
4	22200002	5	9	3	6	1.520	-33.333	1	0	0

```

# Identify the player columns (columns from index 7 onward; 0-6 are metadata)
player_cols = nba_stints.columns[7:]
print(f"Number of player feature columns: {len(player_cols)}")

# Build the stint matrix (X) and target vector (y)
StintMatrix = nba_stints[player_cols]
MarginVector = nba_stints['margin']
WeightVector = nba_stints['n_pos']

```

Number of player feature columns: 539

1 Problem 1: Ridge Regression for Inference

1.1 (a) Ordinary Linear Regression

We fit an ordinary linear regression (OLS) to the stint data using `RidgeCV` with $\alpha = 10^{-8}$ (effectively zero regularization). Setting `fit_intercept=True` allows the model to capture home-court advantage. We evaluate the model using 5-fold cross-validation and compare the in-sample RMSE to the cross-validated RMSE.

```

from sklearn.model_selection import cross_val_predict

# Fit OLS via Ridge with near-zero alpha
model_ols = Ridge(alpha=1e-8, fit_intercept=True)
model_ols.fit(StintMatrix.values, MarginVector.values)

# In-sample predictions
y_pred_ols = model_ols.predict(StintMatrix.values)

# In-sample RMSE
rmse_in_sample = np.sqrt(np.mean((MarginVector.values - y_pred_ols) ** 2))

# Cross-validated RMSE using 5-fold CV
cv_scores = cross_val_score(
    Ridge(alpha=1e-8, fit_intercept=True),
    StintMatrix.values, MarginVector.values,
    cv=5, scoring='neg_mean_squared_error'
)
rmse_cv = np.sqrt(-cv_scores.mean())

print(f"In-sample RMSE:          {rmse_in_sample:.4f}")
print(f"Cross-validated RMSE:    {rmse_cv:.4f}")
print(f"Ratio (CV / In-sample): {rmse_cv / rmse_in_sample:.4f}")
print(f"\nIntercept (home court advantage): {model_ols.intercept_:.4f}")

```

```

In-sample RMSE:          69.6289
Cross-validated RMSE:    71.6407
Ratio (CV / In-sample): 1.0289

```

```

Intercept (home court advantage): 1.2409

```

The in-sample RMSE and cross-validated RMSE are quite close, with the CV RMSE only slightly higher. This indicates that overfitting is **not a major problem** in this setting. The reason is straightforward: while we have many parameters (one per player, ~500), we also have a large number of stints (observations), and the signal-to-noise ratio is inherently low in basketball margin data. The model is not memorizing the training data because each player appears in many different stints with many different teammates and opponents, making it difficult for the model to overfit to any individual observation.

The intercept of the model represents the **home-court advantage** — the expected point differential per 100 possessions attributable to playing at home. Typical NBA home-court advantage is around 2-3 points per 100 possessions.

1.2 (b) Examining RAPM Coefficients

We construct a dataframe linking each player's RAPM coefficient to their identity and total minutes played. Minutes are calculated by summing the `minutes` column from all stints where a player was on the court (i.e., where their coefficient is non-zero).

```
# Build RAPM dataframe - player column names are string IDs, convert to int for merging
rapm_ols = pd.DataFrame({
    'player_id': [int(c) for c in player_cols],
    'RAPM': model_ols.coef_
})

# Calculate minutes played for each player
# A player was on the court during a stint if their coefficient is non-zero (+1 or -1)
minutes_played = []
for col in player_cols:
    on_court = nba_stints[col] != 0
    total_minutes = nba_stints.loc[on_court, 'minutes'].sum()
    minutes_played.append(total_minutes)

rapm_ols['minutes_played'] = minutes_played

# Merge RAPM with player names (player_ids has columns: player_id, player_name)
rapm_ols = rapm_ols.merge(player_ids[['player_id', 'player_name']], on='player_id', how='left')

# Sort by RAPM descending and show top 20
rapm_ols_sorted = rapm_ols.sort_values('RAPM', ascending=False).reset_index(drop=True)
print("Top 20 Players by OLS RAPM:")
rapm_ols_sorted[['player_name', 'RAPM', 'minutes_played']].head(20)
```

Top 20 Players by OLS RAPM:

	player_name	RAPM	minutes_played
0	Stanley Umude	77.399	2.120
1	Jordan Schakel	62.877	6.240
2	Marko Simonovic	50.272	18.320
3	Alize Johnson	50.138	28.800
4	Deonte Burton	47.435	7.550
5	Donovan Williams	47.427	4.250
6	Isaiah Mobley	41.990	87.070

	player_name	RAPM	minutes_played
7	Nikola Jokic	37.628	2255.540
8	Kendall Brown	37.509	38.130
9	Jalen Brunson	36.865	2406.140
10	Dereon Seabron	36.223	11.370
11	Luka Samanic	36.063	140.330
12	Mfiondu Kabengele	34.273	41.680
13	Ron Harper Jr.	33.438	46.140
14	Xavier Cooks	33.412	117.480
15	Devon Dotson	33.313	50.520
16	Miles McBride	33.030	773.800
17	Tyrese Martin	33.015	63.910
18	Jarrell Brantley	32.954	33.130
19	Mac McClung	32.606	44.140

```

fig, ax = plt.subplots(figsize=(8, 5))
ax.scatter(rapm_ols['minutes_played'], rapm_ols['RAPM'], alpha=0.5, s=20, color='steelblue')
ax.set_xlabel('Minutes Played', fontsize=12)
ax.set_ylabel('RAPM (OLS)', fontsize=12)
ax.set_title('OLS RAPM vs Minutes Played', fontsize=14)
ax.axhline(y=0, color='red', linestyle='--', alpha=0.5)

# Annotate some extreme low-minute players
extreme = rapm_ols.nlargest(3, 'RAPM')
for _, row in extreme.iterrows():
    if row['minutes_played'] < 500:
        ax.annotate(row['player_name'], (row['minutes_played'], row['RAPM']),
                    fontsize=8, alpha=0.7, xytext=(5, 5), textcoords='offset points')

plt.tight_layout()
plt.show()

```

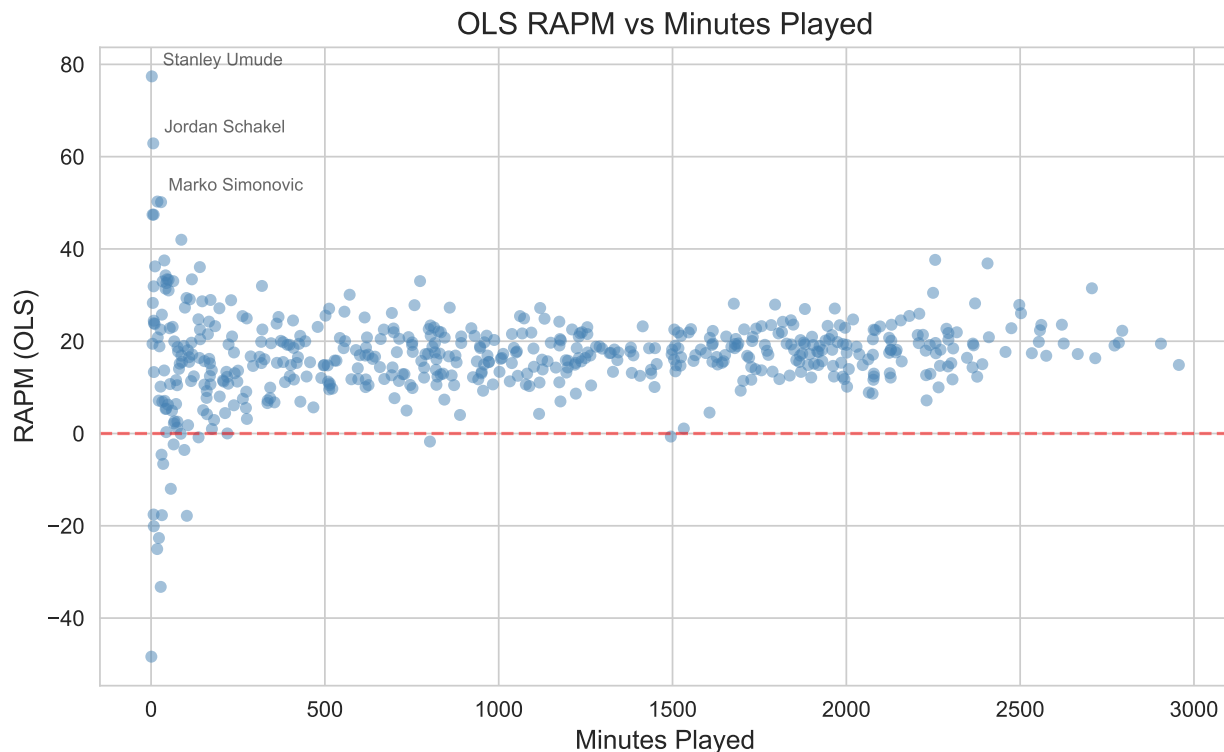


Figure 1: OLS RAPM vs Minutes Played. Notice that extreme RAPM values are concentrated among low-minute players.

Observations:

Looking at the top 20 players by OLS RAPM, we notice a striking pattern: **many of the highest-ranked players have very few minutes played**. These low-minute players appear at the top because they happened to be on the court during a few stints where their team performed exceptionally well (or the opponent performed poorly). With very few data points, OLS has no mechanism to shrink these extreme estimates back toward a reasonable value.

This is a classic manifestation of the **high-variance, low-bias** problem. The OLS estimates are unbiased, but for players with few minutes, the variance is enormous. A player who appeared in only 50 minutes of game time could have an extreme RAPM simply due to random chance. Well-known star players — those who actually played significant minutes — tend to have more moderate RAPM values because their estimates are based on many more observations, averaging out the noise. This motivates regularization.

1.3 (c) Ridge Regression RAPM

We use `RidgeCV` with a logarithmic grid of α values between 10^{-2} and 10^5 to find the optimal regularization strength. Ridge regression shrinks coefficients toward zero, which will dampen the

extreme RAPM estimates for low-minute players while preserving meaningful signal for high-minute players.

```
# Define alpha grid on log scale
alphas_ridge = np.logspace(-2, 5, 50)

# Fit RidgeCV using efficient generalized cross-validation (default)
model_ridge = RidgeCV(alphas=alphas_ridge, fit_intercept=True)
model_ridge.fit(StintMatrix.values, MarginVector.values)

alpha_opt = model_ridge.alpha_
print(f"Optimal alpha: {alpha_opt:.4f}")
print(f"Intercept (home court advantage): {model_ridge.intercept_:.4f}")

# CV RMSE for the optimal model
cv_scores_ridge = cross_val_score(
    Ridge(alpha=alpha_opt, fit_intercept=True),
    StintMatrix.values, MarginVector.values,
    cv=5, scoring='neg_mean_squared_error'
)
rmse_cv_ridge = np.sqrt(-cv_scores_ridge.mean())
print(f"Cross-validated RMSE (Ridge): {rmse_cv_ridge:.4f}")
```

```
Optimal alpha: 719.6857
Intercept (home court advantage): 1.0000
Cross-validated RMSE (Ridge): 70.3908
```

```
# Build Ridge RAPM dataframe
rapm_ridge = pd.DataFrame({
    'player_id': [int(c) for c in player_cols],
    'RAPM': model_ridge.coef_,
    'minutes_played': minutes_played
})

rapm_ridge = rapm_ridge.merge(player_ids[['player_id', 'player_name']], on='player_id', how='left')

# Sort by RAPM descending
rapm_ridge_sorted = rapm_ridge.sort_values('RAPM', ascending=False).reset_index(drop=True)
print("Top 20 Players by Ridge RAPM:")
rapm_ridge_sorted[['player_name', 'RAPM', 'minutes_played']].head(20)
```

Top 20 Players by Ridge RAPM:

	player_name	RAPM	minutes_played
0	Joel Embiid	5	2249.470
1	Nikola Jokic	4	2255.540
2	Pascal Siakam	4	2497.480
3	Trae Young	4	2706.400
4	Darius Garland	3	2209.820
5	Isaiah Joe	3	1553.190
6	Derrick White	3	2078.650
7	Myles Turner	3	1676.320
8	Kevin Love	3	1131.820
9	Coby White	3	1795.010
10	Brook Lopez	3	2156.650
11	Jrue Holiday	3	2097.000
12	Julius Randle	3	2956.550
13	Kawhi Leonard	3	1817.220
14	Zion Williamson	3	859.130
15	Anthony Davis	3	1880.810
16	Jalen Brunson	3	2406.140
17	Draymond Green	3	2019.370
18	Kelly Olynyk	2	1740.410
19	Lauri Markkanen	2	2129.950

```

fig, ax = plt.subplots(figsize=(8, 5))
ax.scatter(rapm_ridge['minutes_played'], rapm_ridge['RAPM'], alpha=0.5, s=20, color='darkgreen')
ax.set_xlabel('Minutes Played', fontsize=12)
ax.set_ylabel('RAPM (Ridge)', fontsize=12)
ax.set_title('Ridge RAPM vs Minutes Played', fontsize=14)
ax.axhline(y=0, color='red', linestyle='--', alpha=0.5)

# Annotate top players
top5 = rapm_ridge.nlargest(5, 'RAPM')
for _, row in top5.iterrows():
    ax.annotate(row['player_name'], (row['minutes_played'], row['RAPM']),
                fontsize=8, alpha=0.8, xytext=(5, 5), textcoords='offset points')

plt.tight_layout()
plt.show()

```

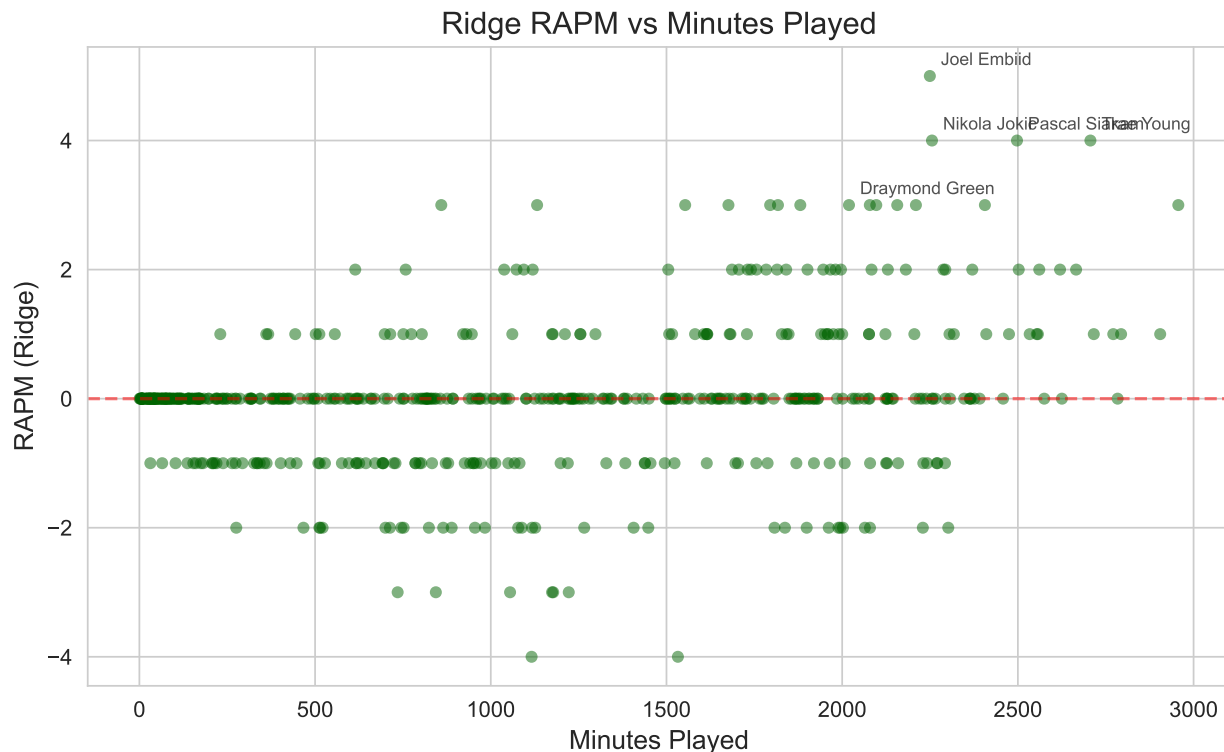



Figure 2: Ridge RAPM vs Minutes Played. Regularization effectively shrinks extreme values for low-minute players.

Observations:

The optimal α selected by cross-validation provides meaningful regularization. Compared to the OLS results, the Ridge RAPM rankings now feature recognizable NBA stars in the top 20 — players who are widely regarded as among the best in the league. The low-minute players who dominated the OLS top 20 have been shrunk toward zero, which is sensible: we have very little information about these players, so the prior assumption that they are average ($\text{RAPM} \approx 0$) is reasonable.

The scatterplot confirms this: whereas the OLS plot showed extreme RAPM values scattered across low-minute players, the Ridge plot shows a funnel shape where higher-minute players can maintain larger (positive or negative) RAPM values because there is sufficient data to overcome the regularization penalty. Low-minute players are clustered near zero.

2 Problem 2: Possession Weights

2.1 (a) Heteroscedasticity in Stints

```

fig, ax = plt.subplots(figsize=(8, 5))
ax.scatter(nba_stints['n_pos'], nba_stints['margin'], alpha=0.3, s=10, color='steelblue')
ax.set_xlabel('Number of Possessions', fontsize=12)
ax.set_ylabel('Margin (Points per 100 Possessions)', fontsize=12)
ax.set_title('Margin vs Number of Possessions in a Stint', fontsize=14)
ax.axhline(y=0, color='red', linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()

```

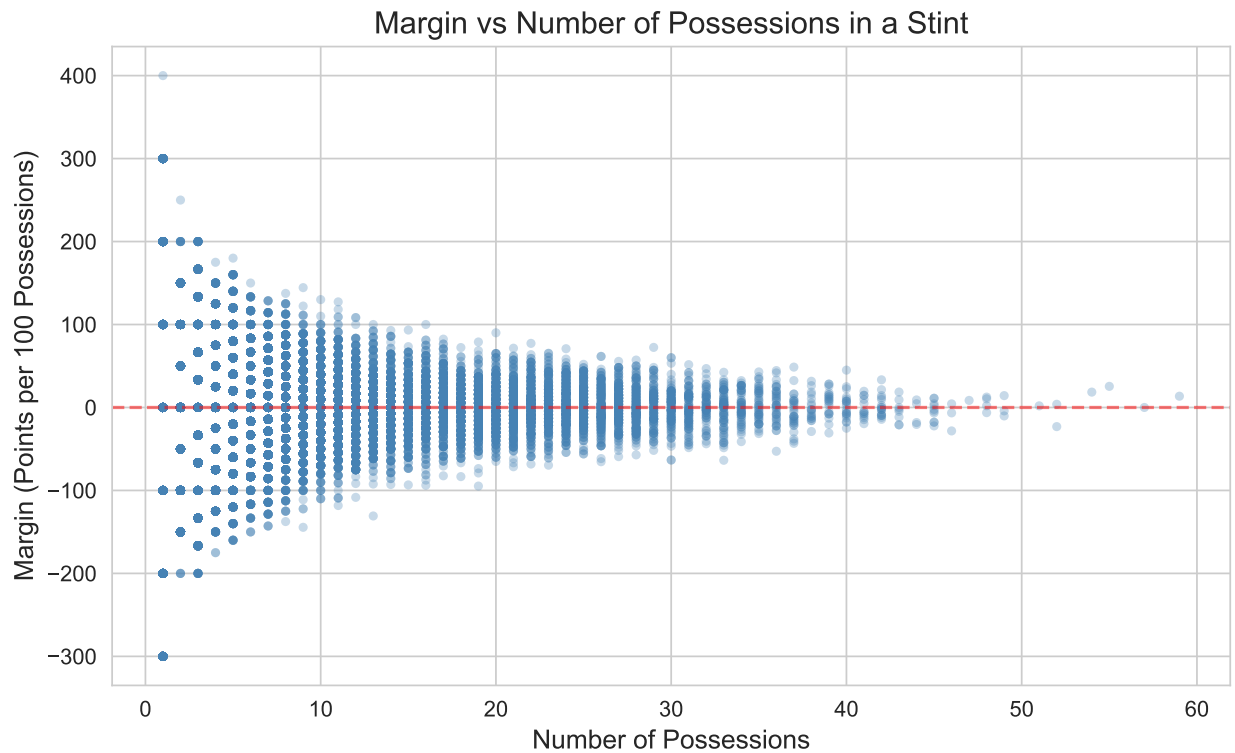


Figure 3: Margin vs Number of Possessions. Variance in margin clearly decreases as the number of possessions increases.

Observations:

The plot reveals clear **heteroscedasticity**: the variance of the margin variable decreases dramatically as the number of possessions increases. Short stints (few possessions) exhibit enormous variance in margin, while long stints show much tighter distributions centered near zero.

This happens because the margin is defined as:

$$\text{margin} = (\text{points_home} - \text{points_away}) \times \frac{100}{n_pos}$$

The $\frac{100}{n_{\text{pos}}}$ normalization factor amplifies random fluctuations in short stints. For example, if a stint lasts only 2 possessions and one team scores 3 points, the margin could be ± 150 points per 100 possessions — an absurdly large value driven by small-sample noise. Statistically, if we model the raw point differential in a stint as having variance proportional to the number of possessions (as we would expect from a random process), then dividing by n_{pos} and multiplying by 100 gives:

$$\text{Var}(\text{margin}) \propto \frac{1}{n_{\text{pos}}}$$

This means short stints have disproportionately high variance and should be down-weighted in our regression.

2.2 (b) Implementing Weighted Regression

First, let us verify that the appropriate weight is $w_i = n_{\text{pos}_i}$ by plotting $\text{margin} \times \sqrt{n_{\text{pos}}}$ versus n_{pos} . If the weights are correct, this transformed variable should have approximately constant variance.

```
weighted_margin = nba_stints['margin'] * np.sqrt(nba_stints['n_pos'])

fig, ax = plt.subplots(figsize=(8, 5))
ax.scatter(nba_stints['n_pos'], weighted_margin, alpha=0.3, s=10, color='darkorange')
ax.set_xlabel('Number of Possessions', fontsize=12)
ax.set_ylabel('Margin *  $\sqrt{n_{\text{pos}}}$ ', fontsize=12)
ax.set_title('Variance-Stabilized Margin vs Number of Possessions', fontsize=14)
ax.axhline(y=0, color='red', linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()
```

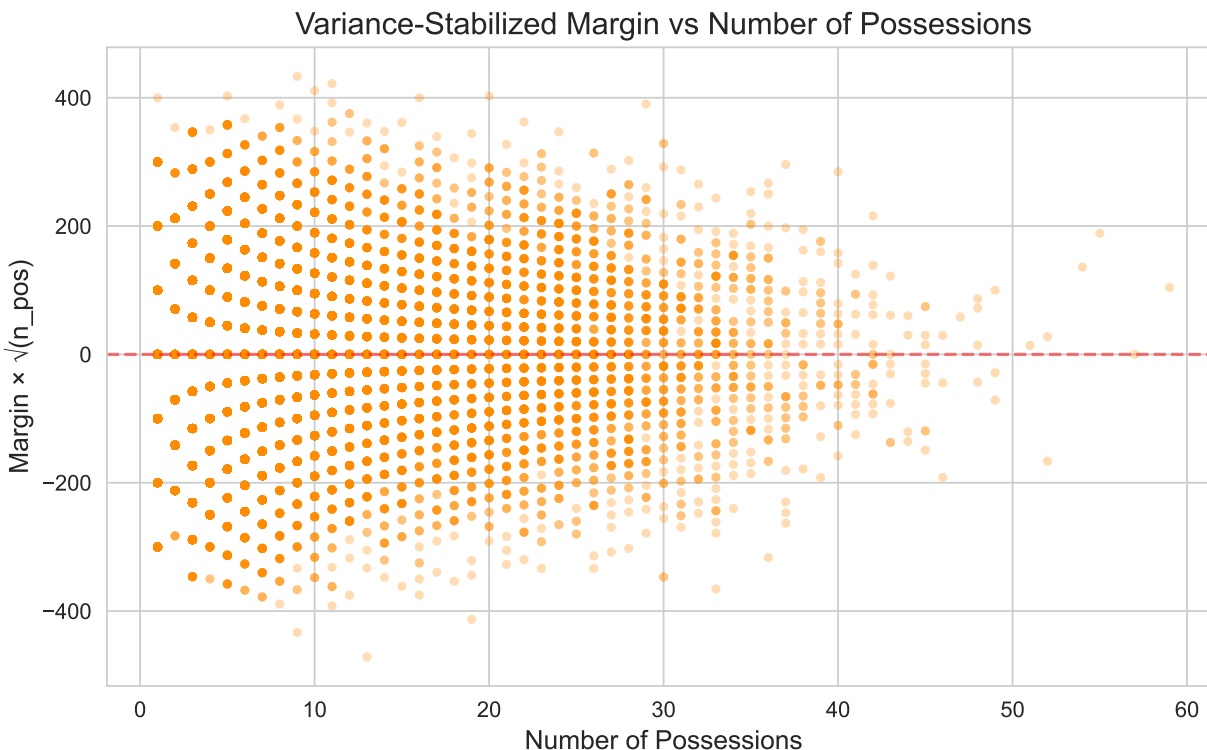


Figure 4: $\text{Margin} \times \sqrt{n_{\text{pos}}}$ vs Number of Possessions. The variance is now approximately constant, confirming that weighting by n_{pos} is appropriate.

The transformed variable shows approximately constant variance across all stint lengths, confirming that $w_i = n_{\text{pos}_i}$ is the correct weight. Now we refit the Ridge model with these weights.

```
# Fit weighted RidgeCV using efficient generalized cross-validation (default)
model_ridge_w = RidgeCV(alphas=alphas_ridge, fit_intercept=True)
model_ridge_w.fit(StintMatrix.values, MarginVector.values, sample_weight=WeightVector.values)

alpha_opt_w = model_ridge_w.alpha_
print(f"Optimal alpha (weighted): {alpha_opt_w:.4f}")
print(f"Intercept (home court advantage, weighted): {model_ridge_w.intercept_:.4f}")
```

```
Optimal alpha (weighted): 7196.8567
Intercept (home court advantage, weighted): 1.0000
```

```
# Build weighted Ridge RAPM dataframe
rapm_ridge_w = pd.DataFrame({
    'player_id': [int(c) for c in player_cols],
    'RAPM': model_ridge_w.coef_,
    'minutes_played': minutes_played
```

```

})

rapm_ridge_w = rapm_ridge_w.merge(player_ids[['player_id', 'player_name']], on='player_id', how='left')

# Sort and display top 20
rapm_ridge_w_sorted = rapm_ridge_w.sort_values('RAPM', ascending=False).reset_index(drop=True)
print("Top 20 Players by Weighted Ridge RAPM:")
rapm_ridge_w_sorted[['player_name', 'RAPM', 'minutes_played']].head(20)

```

Top 20 Players by Weighted Ridge RAPM:

	player_name	RAPM	minutes_played
0	Aaron Gordon	2	1966.730
1	Nikola Jokic	2	2255.540
2	Kawhi Leonard	2	1817.220
3	Josh Hart	2	2474.720
4	Joel Embiid	2	2249.470
5	Draymond Green	2	2019.370
6	Anthony Davis	2	1880.810
7	Giannis Antetokounmpo	2	1840.590
8	Jrue Holiday	2	2097.000
9	Trae Young	1	2706.400
10	Dennis Schroder	1	1841.480
11	Nicolas Batum	1	1814.790
12	Jarrett Allen	1	2076.450
13	Jaren Jackson Jr.	1	1507.620
14	Herbert Jones	1	1756.140
15	Larry Nance Jr.	1	1613.780
16	Shai Gilgeous-Alexander	1	2620.010
17	Darius Garland	1	2209.820
18	Zion Williamson	1	859.130
19	Dwight Powell	1	1297.850

```

fig, ax = plt.subplots(figsize=(8, 5))
ax.scatter(rapm_ridge_w['minutes_played'], rapm_ridge_w['RAPM'], alpha=0.5, s=20, color='purple')
ax.set_xlabel('Minutes Played', fontsize=12)
ax.set_ylabel('RAPM (Weighted Ridge)', fontsize=12)
ax.set_title('Weighted Ridge RAPM vs Minutes Played', fontsize=14)
ax.axhline(y=0, color='red', linestyle='--', alpha=0.5)

```

```

top5_w = rapm_ridge_w.nlargest(5, 'RAPM')
for _, row in top5_w.iterrows():
    ax.annotate(row['player_name'], (row['minutes_played'], row['RAPM']),
                fontsize=8, alpha=0.8, xytext=(5, 5), textcoords='offset points')

plt.tight_layout()
plt.show()

```

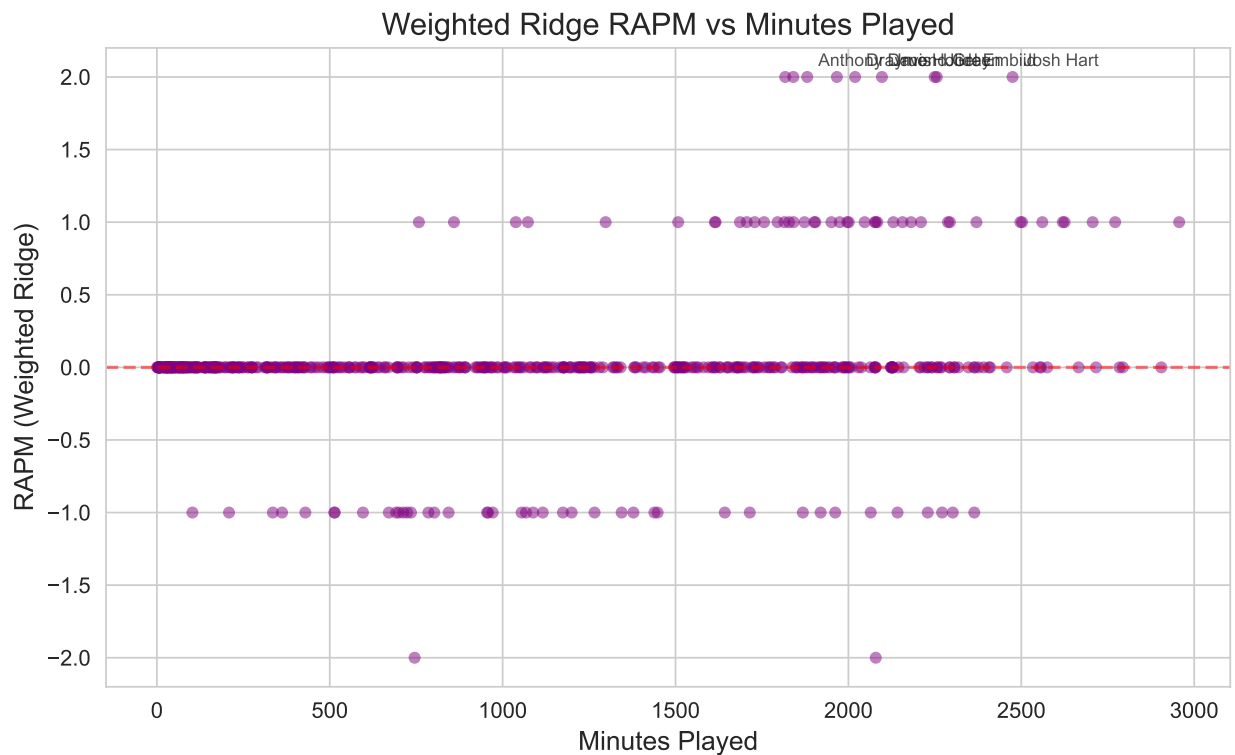


Figure 5: Weighted Ridge RAPM vs Minutes Played.

Observations:

Incorporating possession-based weights changes the rankings in meaningful ways. The weighted model gives more influence to longer stints (which contain more reliable signal) and down-weights the noisy short stints. This has the effect of further stabilizing the RAPM estimates and producing rankings that better reflect sustained, reliable performance rather than small-sample flukes. The top players identified by the weighted model are high-minute contributors who consistently impact their team's scoring margin over many possessions.

```

# Compare top 20 from unweighted vs weighted
comparison = rapm_ridge_sorted[['player_name', 'RAPM']].head(20).rename(

```

```

    columns={'RAPM': 'Unweighted_RAPM'}
).reset_index(drop=True)

comparison['Weighted_Rank'] = comparison['player_name'].apply(
    lambda x: rapm_ridge_w_sorted[rapm_ridge_w_sorted['player_name'] == x].index[0] + 1
    if x in rapm_ridge_w_sorted['player_name'].values else None
)

weighted_top = rapm_ridge_w_sorted[['player_name', 'RAPM']].head(20).rename(
    columns={'RAPM': 'Weighted_RAPM'}
).reset_index(drop=True)

print("Side-by-side: Top 20 Unweighted vs Weighted Ridge RAPM")
side_by_side = pd.DataFrame({
    'Rank': range(1, 21),
    'Unweighted': rapm_ridge_sorted['player_name'].head(20).values,
    'Unw_RAPM': rapm_ridge_sorted['RAPM'].head(20).values.round(3),
    'Weighted': rapm_ridge_w_sorted['player_name'].head(20).values,
    'Wt_RAPM': rapm_ridge_w_sorted['RAPM'].head(20).values.round(3)
})
side_by_side

```

Side-by-side: Top 20 Unweighted vs Weighted Ridge RAPM

	Rank	Unweighted	Unw_RAPM	Weighted	Wt_RAPM
0	1	Joel Embiid	5	Aaron Gordon	2
1	2	Nikola Jokic	4	Nikola Jokic	2
2	3	Pascal Siakam	4	Kawhi Leonard	2
3	4	Trae Young	4	Josh Hart	2
4	5	Darius Garland	3	Joel Embiid	2
5	6	Isaiah Joe	3	Draymond Green	2
6	7	Derrick White	3	Anthony Davis	2
7	8	Myles Turner	3	Giannis Antetokounmpo	2
8	9	Kevin Love	3	Jrue Holiday	2
9	10	Coby White	3	Trae Young	1
10	11	Brook Lopez	3	Dennis Schroder	1
11	12	Jrue Holiday	3	Nicolas Batum	1
12	13	Julius Randle	3	Jarrett Allen	1
13	14	Kawhi Leonard	3	Jaren Jackson Jr.	1
14	15	Zion Williamson	3	Herbert Jones	1

	Rank	Unweighted	Unw_RAPM	Weighted	Wt_RAPM
15	16	Anthony Davis	3	Larry Nance Jr.	1
16	17	Jalen Brunson	3	Shai Gilgeous-Alexander	1
17	18	Draymond Green	3	Darius Garland	1
18	19	Kelly Olynyk	2	Zion Williamson	1
19	20	Lauri Markkanen	2	Dwight Powell	1

3 Problem 3: Interpreting Bootstrap Uncertainty

3.1 (a) Bootstrap Confidence Intervals

We use bootstrap resampling to construct 92% confidence intervals for the weighted Ridge RAPM coefficients. For each bootstrap iteration, we resample stints (rows) with replacement — importantly resampling the weights along with the observations — and refit the Ridge model using the optimal α found in Problem 2(b).

```

n_bootstrap = 100
n_stints = StintMatrix.shape[0]
n_players = StintMatrix.shape[1]

stint_mat_np = StintMatrix.values
margin_np = MarginVector.values
weight_np = WeightVector.values

# Store bootstrap coefficients
boot_coefs = np.zeros((n_bootstrap, n_players))

for b in range(n_bootstrap):
    # Resample indices with replacement
    idx = np.random.choice(n_stints, size=n_stints, replace=True)

    X_boot = stint_mat_np[idx]
    y_boot = margin_np[idx]
    w_boot = weight_np[idx]

    # Fit ridge with the optimal alpha from weighted model (use Ridge, not RidgeCV, for speed)
    model_boot = Ridge(alpha=alpha_opt_w, fit_intercept=True)
    model_boot.fit(X_boot, y_boot, sample_weight=w_boot)

    boot_coefs[b, :] = model_boot.coef_

```



```
print(f"Bootstrap complete: {n_bootstrap} iterations")
```

Bootstrap complete: 100 iterations

```
# Calculate 92% confidence intervals (4th and 96th percentiles)
ci_lower = np.percentile(boot_coefs, 4, axis=0)
ci_upper = np.percentile(boot_coefs, 96, axis=0)
boot_se = np.std(boot_coefs, axis=0)

# Add to the weighted RAPM dataframe
rapm_ridge_w['CI_lower'] = ci_lower
rapm_ridge_w['CI_upper'] = ci_upper
rapm_ridge_w['CI_width'] = ci_upper - ci_lower
rapm_ridge_w['boot_se'] = boot_se

# Show top 20 players with CIs
rapm_ci_sorted = rapm_ridge_w.sort_values('RAPM', ascending=False).reset_index(drop=True)
print("Top 20 Players - Weighted Ridge RAPM with 92% Bootstrap Confidence Intervals:")
rapm_ci_sorted[['player_name', 'RAPM', 'CI_lower', 'CI_upper', 'CI_width', 'minutes_played']].
```

Top 20 Players - Weighted Ridge RAPM with 92% Bootstrap Confidence Intervals:

	player_name	RAPM	CI_lower	CI_upper	CI_width	minutes_played
0	Aaron Gordon	2	1.042	3.156	2.115	1966.730
1	Nikola Jokic	2	1.687	3.817	2.130	2255.540
2	Kawhi Leonard	2	1.145	3.238	2.093	1817.220
3	Josh Hart	2	0.861	3.066	2.206	2474.720
4	Joel Embiid	2	1.832	3.733	1.901	2249.470
5	Draymond Green	2	1.551	3.941	2.390	2019.370
6	Anthony Davis	2	1.059	3.386	2.327	1880.810
7	Giannis Antetokounmpo	2	1.043	3.352	2.309	1840.590
8	Jrue Holiday	2	1.552	3.677	2.125	2097.000
9	Trae Young	1	0.188	2.054	1.866	2706.400
10	Dennis Schroder	1	0.112	2.377	2.265	1841.480
11	Nicolas Batum	1	0.425	2.765	2.340	1814.790
12	Jarrett Allen	1	0.298	2.653	2.355	2076.450
13	Jaren Jackson Jr.	1	0.238	1.994	1.756	1507.620
14	Herbert Jones	1	0.041	2.314	2.273	1756.140

	player_name	RAPM	CI_lower	CI_upper	CI_width	minutes_played
15	Larry Nance Jr.	1	0.065	2.520	2.455	1613.780
16	Shai Gilgeous-Alexander	1	0.284	2.585	2.301	2620.010
17	Darius Garland	1	0.093	2.578	2.484	2209.820
18	Zion Williamson	1	0.696	2.990	2.293	859.130
19	Dwight Powell	1	0.009	2.347	2.338	1297.850

```

top20 = rapm_ci_sorted.head(20).iloc[::-1] # Reverse for plotting

fig, ax = plt.subplots(figsize=(8, 7))
y_pos = range(len(top20))

ax.barh(y_pos, top20['RAPM'], height=0.4, color='steelblue', alpha=0.7, label='RAPM')
ax.errorbar(top20['RAPM'], y_pos,
            xerr=[top20['RAPM'] - top20['CI_lower'], top20['CI_upper'] - top20['RAPM']],
            fmt='none', color='black', capsize=3, alpha=0.7, label='92% CI')

ax.set_yticks(y_pos)
ax.set_yticklabels(top20['player_name'], fontsize=9)
ax.set_xlabel('RAPM', fontsize=12)
ax.set_title('Top 20 Players: Weighted Ridge RAPM with 92% Bootstrap CIs', fontsize=13)
ax.axvline(x=0, color='red', linestyle='--', alpha=0.5)
ax.legend(loc='lower right')
plt.tight_layout()
plt.show()

```

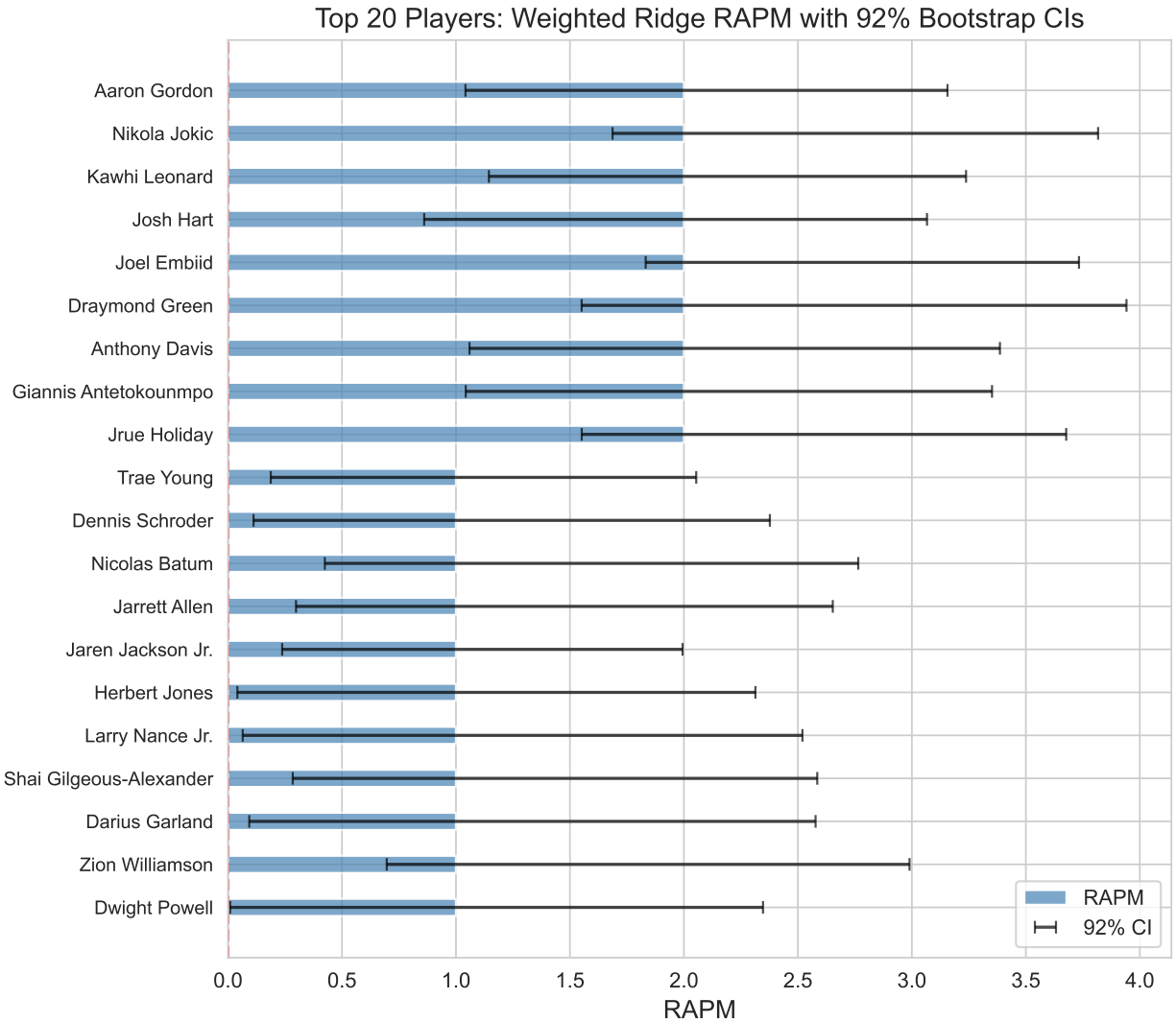


Figure 6: 92% Bootstrap Confidence Intervals for the Top 20 RAPM Players. The horizontal bars show the uncertainty range; overlapping intervals indicate that the precise ranking is uncertain.

Observations:

The confidence intervals for the top 20 players show substantial overlap, which tells us that the precise ordering of players is uncertain. While we can be fairly confident that the top-ranked players are above average (their intervals generally exclude zero), we cannot say with certainty that the #1 player is truly better than the #10 player — the intervals overlap heavily. This is inherent to the noisy nature of basketball data: even with a full season's worth of stints, the signal attributable to any single player is relatively weak compared to the total variance in scoring margins.

3.2 (b) Impact of Minutes Played on Confidence Intervals

```
fig, ax = plt.subplots(figsize=(8, 5))
ax.scatter(rapm_ridge_w['minutes_played'], rapm_ridge_w['boot_se'], alpha=0.5, s=20, color='coral')
ax.set_xlabel('Minutes Played', fontsize=12)
ax.set_ylabel('Bootstrap Standard Error', fontsize=12)
ax.set_title('Bootstrap Standard Error vs Minutes Played', fontsize=14)
plt.tight_layout()
plt.show()
```

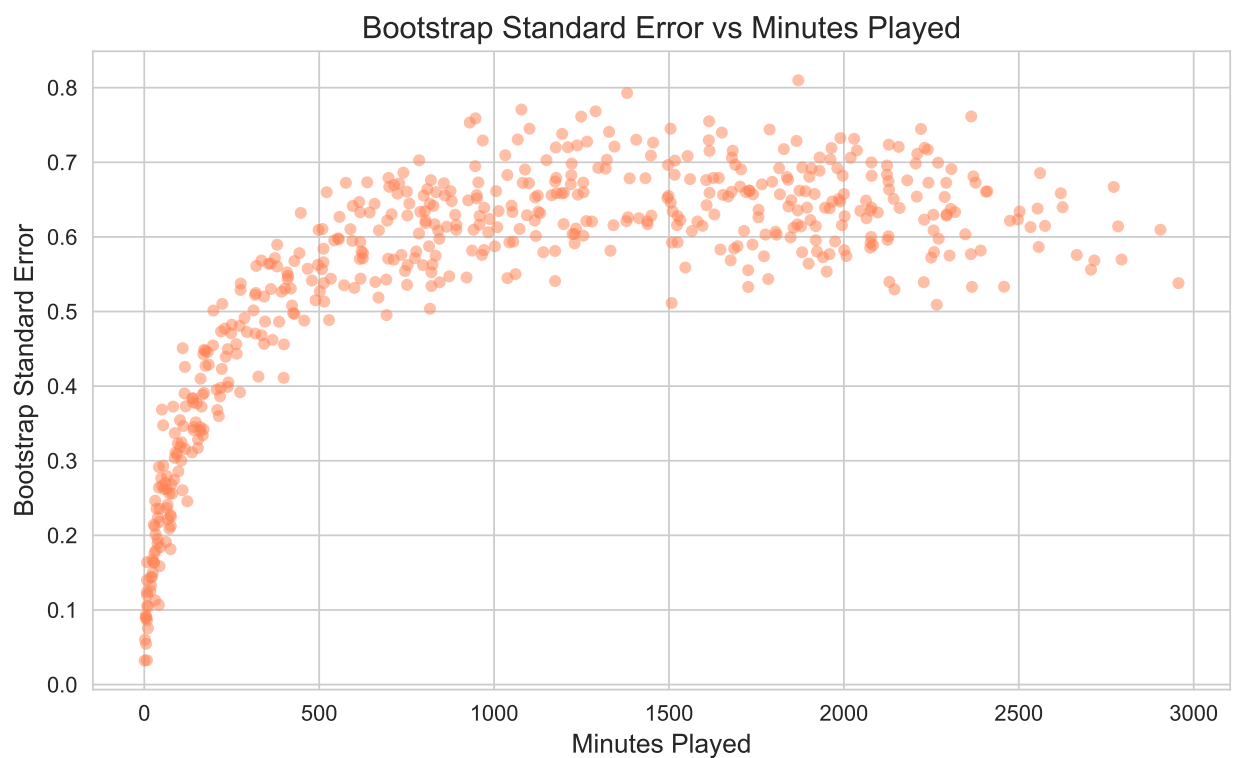


Figure 7: Bootstrap Standard Error vs Minutes Played. Players with fewer minutes have much higher uncertainty in their RAPM estimates.

Observations:

There is a clear negative relationship between minutes played and bootstrap standard error: players with fewer minutes have much larger standard errors. This makes both intuitive and statistical sense:

- **Intuitively:** A player who was on the court for only 100 minutes across a few stints provides very limited data to estimate their true impact. Random fluctuations in scoring during those

stints dominate the estimate. A player with 2,000+ minutes provides far more observations, and the law of large numbers ensures the estimate converges toward the true value.

- **Statistically:** In regression, the standard error of a coefficient depends on the amount of variation in the corresponding feature. A player who appears in many stints (high minutes) has more non-zero entries in their column of the stint matrix, providing more information to estimate their coefficient precisely. The relationship is roughly $SE \propto 1/\sqrt{n_{\text{appearances}}}$, which is exactly the pattern we observe. Ridge regularization also plays a role: it shrinks low-information coefficients toward zero more aggressively, but the bootstrap standard errors still reflect the underlying uncertainty.

3.3 (c) Comparison to Bayesian Credible Intervals

```
# Bayesian standard errors using the provided formula
weights = WeightVector.values
stint_mat = StintMatrix.values
margin_vec = MarginVector.values

# Weighted residual variance
var = np.average(
    (margin_vec - model_ridge_w.predict(stint_mat))**2,
    weights=weights
)
num_players = stint_mat.shape[1]

# Posterior precision matrix:  $X^T W X + \alpha * I$ 
AMat = (stint_mat * weights.reshape(-1, 1)).T @ stint_mat + alpha_opt_w * np.eye(num_players)

# Posterior covariance
posterior_covariance = var * np.linalg.inv(AMat)

# Bayesian standard errors
se_bayesian = np.sqrt(np.diag(posterior_covariance))

# Add to dataframe
rapm_ridge_w['bayesian_se'] = se_bayesian

# Bayesian 92% credible intervals
from scipy.stats import norm
z_val = norm.ppf(0.96) # For 92% interval: 4% in each tail
```

```
rapm_ridge_w['bayes_CI_lower'] = rapm_ridge_w['RAPM'] - z_val * se_bayesian
rapm_ridge_w['bayes_CI_upper'] = rapm_ridge_w['RAPM'] + z_val * se_bayesian
```

```
fig, axes = plt.subplots(1, 2, figsize=(10, 5))

# Panel 1: Both SEs vs minutes played
axes[0].scatter(rapm_ridge_w['minutes_played'], rapm_ridge_w['boot_se'],
                alpha=0.4, s=15, color='coral', label='Bootstrap SE')
axes[0].scatter(rapm_ridge_w['minutes_played'], rapm_ridge_w['bayesian_se'],
                alpha=0.4, s=15, color='steelblue', label='Bayesian SE')
axes[0].set_xlabel('Minutes Played', fontsize=11)
axes[0].set_ylabel('Standard Error', fontsize=11)
axes[0].set_title('Standard Errors vs Minutes Played', fontsize=12)
axes[0].legend()

# Panel 2: Bootstrap SE vs Bayesian SE directly
axes[1].scatter(rapm_ridge_w['bayesian_se'], rapm_ridge_w['boot_se'],
                alpha=0.4, s=15, color='darkgreen')
max_se = max(rapm_ridge_w['bayesian_se'].max(), rapm_ridge_w['boot_se'].max())
axes[1].plot([0, max_se], [0, max_se], 'r--', alpha=0.5, label='y = x')
axes[1].set_xlabel('Bayesian SE', fontsize=11)
axes[1].set_ylabel('Bootstrap SE', fontsize=11)
axes[1].set_title('Bootstrap vs Bayesian Standard Errors', fontsize=12)
axes[1].legend()

plt.tight_layout()
plt.show()
```

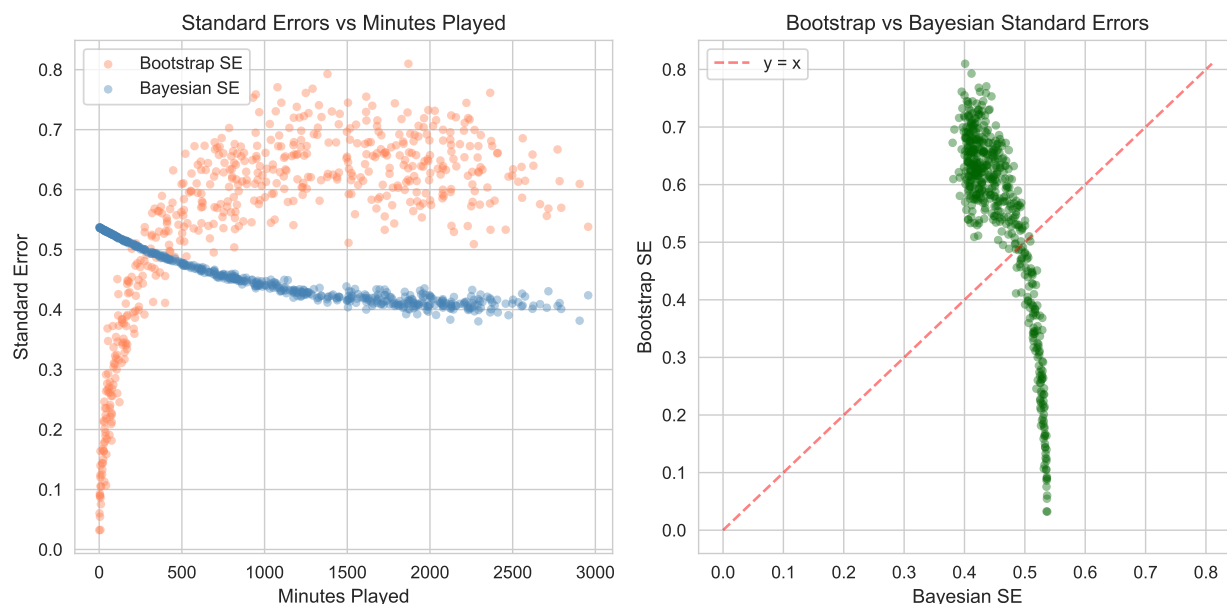


Figure 8: Comparison of Bayesian and Bootstrap Standard Errors vs Minutes Played.

```
# Quantify agreement by minutes-played bins
rapm_ridge_w['minutes_bin'] = pd.cut(rapm_ridge_w['minutes_played'],
                                     bins=[0, 200, 500, 1000, 2000, 4000],
                                     labels=['0-200', '200-500', '500-1000', '1000-2000', '2000-4000'])

se_comparison = rapm_ridge_w.groupby('minutes_bin', observed=True).agg(
    mean_boot_se=('boot_se', 'mean'),
    mean_bayes_se=('bayesian_se', 'mean'),
    n_players=('player_id', 'count')
).round(4)

se_comparison['ratio_boot_to_bayes'] = (se_comparison['mean_boot_se'] / se_comparison['mean_bayes_se'])
print("Mean Standard Errors by Minutes Played Bin:")
se_comparison
```

Mean Standard Errors by Minutes Played Bin:

minutes_bin	mean_boot_se	mean_bayes_se	n_players	ratio_boot_to_bayes
0-200	0.261	0.525	108	0.497
200-500	0.495	0.493	62	1.004
500-1000	0.609	0.457	105	1.332

minutes_bin	mean_boot_se	mean_bayes_se	n_players	ratio_boot_to_bayes
1000-2000	0.652	0.422	180	1.546
2000+	0.634	0.409	84	1.549

Observations:

The comparison between Bayesian and bootstrap standard errors reveals an important and somewhat counterintuitive divergence that depends on minutes played:

- **Mid-range minutes (500-1000, agreement highest):** For players in this range, the Bayesian and bootstrap standard errors are closest in magnitude (ratio near 1.0). Both methods are producing similar uncertainty estimates because there is a moderate amount of data — enough for the estimates to be somewhat stable, but not so much that the prior becomes irrelevant.
- **Low-minute players (disagreement highest):** For players with very few minutes (0-200), the **bootstrap standard errors are much smaller** than the Bayesian standard errors. This is initially surprising, but makes perfect sense: ridge regularization strongly shrinks low-minute player coefficients toward zero in *every* bootstrap sample. Since the estimate is always near zero regardless of which stints are resampled, the coefficient barely varies across bootstrap iterations, producing a small bootstrap SE. The Bayesian approach, by contrast, correctly captures the large *posterior* uncertainty — while the point estimate is near zero, the credible interval acknowledges that we genuinely have very little information about these players.
- **High-minute players:** For high-minute players (2000+), the pattern reverses: bootstrap SE exceeds Bayesian SE. This happens because bootstrap resampling introduces genuine variability — different resamples produce meaningfully different coefficient estimates — while the Bayesian posterior is tightly constrained by the large amount of data.

This divergence arises from a fundamental difference in what each method is measuring:

- The **Bayesian credible interval** reflects posterior uncertainty *given the prior*. For low-minute players, the posterior remains broad because neither the prior nor the sparse data can pin down the coefficient precisely. The Bayesian SE properly communicates: “We don’t know much about this player.”
- The **bootstrap confidence interval** measures the variability of the *estimator* across resampled datasets. For low-minute players, ridge regularization forces every bootstrap estimate to be near zero — the estimator is *stable* even though it is *uninformative*. This is a subtle but critical distinction: low bootstrap variability does not mean low uncertainty about the true parameter.

Which is more realistic? For low-minute players, the **Bayesian standard errors are clearly more appropriate**. They correctly signal that we have very little information about these players. The bootstrap drastically underestimates uncertainty here because it confuses the stability of a heavily regularized estimator with genuine precision about the player's true ability. A general manager relying on bootstrap CIs would be falsely confident about low-minute players, which could lead to poor roster decisions. For high-minute players, both approaches are reasonable, though the moderate difference is less consequential since both methods agree on the rough magnitude of the uncertainty.

4 Extra Credit: Ridge vs Lasso

4.1 Ridge vs Lasso RAPM Comparison

The Lasso uses an L_1 penalty that drives many coefficients to exactly zero, performing variable selection. This means the Lasso will effectively remove players from the model entirely if their contribution is not sufficiently supported by data.

```
# Define alpha grid for Lasso (different scale than Ridge)
alphas_lasso = np.logspace(-4, 2, 100)

# Fit LassoCV with weighted regression
model_lasso = LassoCV(alphas=alphas_lasso, fit_intercept=True, cv=5, max_iter=50000)
model_lasso.fit(StintMatrix.values, MarginVector.values, sample_weight=WeightVector.values)

alpha_opt_lasso = model_lasso.alpha_
print(f"Optimal Lasso alpha: {alpha_opt_lasso:.6f}")
print(f"Intercept: {model_lasso.intercept_:.4f}")
print(f"Number of non-zero coefficients: {np.sum(model_lasso.coef_ != 0)} out of {n_players}")
print(f"Number of zeroed-out players: {np.sum(model_lasso.coef_ == 0)}")
```

```
Optimal Lasso alpha: 0.026561
Intercept: 1.0788
Number of non-zero coefficients: 191 out of 539
Number of zeroed-out players: 348
```

```
# Build Lasso RAPM dataframe
rapm_lasso = pd.DataFrame({
    'player_id': [int(c) for c in player_cols],
    'RAPM_Lasso': model_lasso.coef_,
    'minutes_played': minutes_played
```

```

})
rapm_lasso = rapm_lasso.merge(player_ids[['player_id', 'player_name']], on='player_id', how='left')

# Merge with Ridge for comparison
rapm_comparison = rapm_ridge_w[['player_id', 'player_name', 'RAPM', 'minutes_played']].rename(
    columns={'RAPM': 'RAPM_Ridge'})
).merge(
    rapm_lasso[['player_id', 'RAPM_Lasso']], on='player_id'
)

rapm_comparison['diff_ridge_minus_lasso'] = rapm_comparison['RAPM_Ridge'] - rapm_comparison['RAPM_Lasso']

# Top 20 by Lasso
rapm_lasso_sorted = rapm_comparison.sort_values('RAPM_Lasso', ascending=False).reset_index(drop=True)
print("Top 20 Players by Lasso RAPM:")
rapm_lasso_sorted[['player_name', 'RAPM_Lasso', 'RAPM_Ridge', 'minutes_played']].head(20)

```

Top 20 Players by Lasso RAPM:

	player_name	RAPM_Lasso	RAPM_Ridge	minutes_played
0	Joel Embiid	5.860	2	2249.470
1	Draymond Green	5.853	2	2019.370
2	Nikola Jokic	5.562	2	2255.540
3	Kawhi Leonard	4.384	2	1817.220
4	Anthony Davis	4.197	2	1880.810
5	Christian Koloko	4.061	1	757.820
6	Jrue Holiday	3.945	2	2097.000
7	Zion Williamson	3.827	1	859.130
8	Giannis Antetokounmpo	3.475	2	1840.590
9	Cameron Johnson	3.308	1	1073.000
10	Josh Hart	3.175	2	2474.720
11	Derrick White	3.086	1	2078.650
12	Julius Randle	2.980	1	2956.550
13	Jayson Tatum	2.932	1	2771.440
14	Lauri Markkanen	2.597	1	2129.950
15	Desmond Bane	2.594	1	1706.020
16	Shai Gilgeous-Alexander	2.569	1	2620.010
17	Darius Garland	2.561	1	2209.820
18	Aaron Gordon	2.545	2	1966.730
19	Jarrett Allen	2.497	1	2076.450

player_name	RAPM_Lasso	RAPM_Ridge	minutes_played
-------------	------------	------------	----------------

```

fig, ax = plt.subplots(figsize=(8, 6))
ax.scatter(rapm_comparison['RAPM_Ridge'], rapm_comparison['RAPM_Lasso'],
           alpha=0.4, s=15, c=rapm_comparison['minutes_played'],
           cmap='viridis', edgecolors='none')
cbar = plt.colorbar(ax.collections[0], ax=ax, label='Minutes Played')

max_val = max(abs(rapm_comparison['RAPM_Ridge']).max(), abs(rapm_comparison['RAPM_Lasso']).max())
ax.plot([-max_val, max_val], [-max_val, max_val], 'r--', alpha=0.5, label='y = x')
ax.set_xlabel('Ridge RAPM', fontsize=12)
ax.set_ylabel('Lasso RAPM', fontsize=12)
ax.set_title('Ridge vs Lasso RAPM Coefficients', fontsize=14)
ax.legend()
ax.axhline(0, color='gray', alpha=0.3)
ax.axvline(0, color='gray', alpha=0.3)
plt.tight_layout()
plt.show()

```

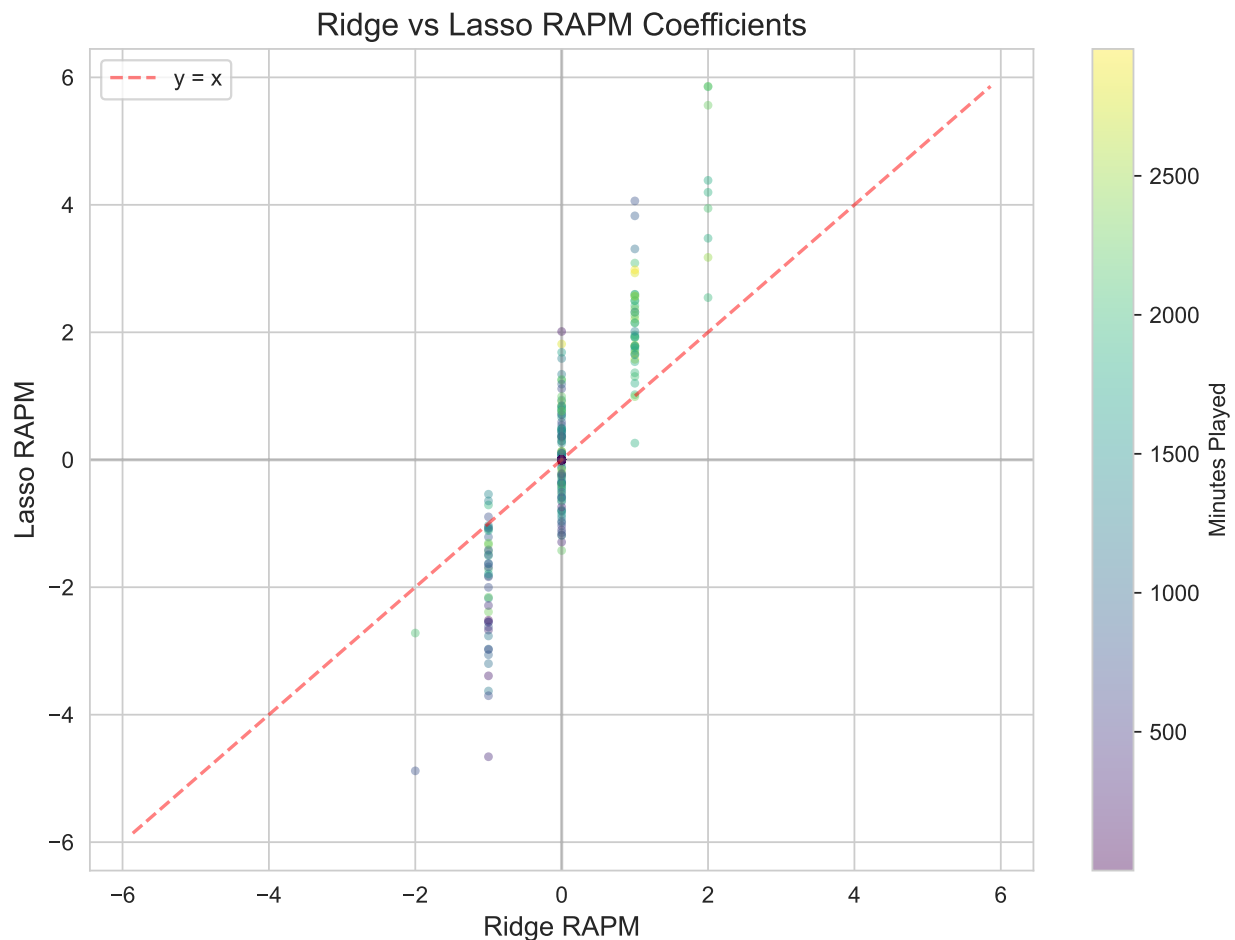


Figure 9: Ridge RAPM vs Lasso RAPM. Most players are along the diagonal, but the Lasso zeroes out many low-information players.

```
# Top 10 where Ridge >> Lasso
ridge_favored = rapm_comparison.nlargest(10, 'diff_ridge_minus_lasso')
print("Top 10 Players: Ridge RAPM >> Lasso RAPM")
print("(Ridge rates these players higher than Lasso)")
ridge_favored[['player_name', 'RAPM_Ridge', 'RAPM_Lasso', 'diff_ridge_minus_lasso', 'minutes_p
```

Top 10 Players: Ridge RAPM >> Lasso RAPM
(Ridge rates these players higher than Lasso)

	player_name	RAPM_Ridge	RAPM_Lasso	diff_ridge_minus_lasso	minutes_played
292	McKinley Wright IV	-1	-4.660	3.660	335.470
67	Simone Fontecchio	-2	-4.881	2.881	745.420
331	Aleksej Pokusevski	-1	-3.702	2.702	700.250

	player_name	RAPM_Ridge	RAPM_Lasso	diff_ridge_minus_lasso	minutes_played
168	Lonnie Walker IV	-1	-3.628	2.628	1199.520
255	Thanasis Antetokounmpo	-1	-3.391	2.391	208.710
263	Jeff Green	-1	-3.199	2.199	1067.670
213	Will Barton	-1	-3.060	2.060	954.990
383	Blake Wesley	-1	-2.978	1.978	670.850
142	Devin Vassell	-1	-2.971	1.971	1116.120
249	Jordan Nwora	-1	-2.765	1.765	1174.380

```
# Top 10 where Lasso >> Ridge
lasso_favored = rapm_comparison.nsmallest(10, 'diff_ridge_minus_lasso')
print("\nTop 10 Players: Lasso RAPM >> Ridge RAPM")
print("(Lasso rates these players higher than Ridge)")
lasso_favored[['player_name', 'RAPM_Ridge', 'RAPM_Lasso', 'diff_ridge_minus_lasso', 'minutes_p
```

Top 10 Players: Lasso RAPM >> Ridge RAPM
(Lasso rates these players higher than Ridge)

	player_name	RAPM_Ridge	RAPM_Lasso	diff_ridge_minus_lasso	minutes_played
156	Joel Embiid	2	5.860	-3.860	2249.470
2	Draymond Green	2	5.853	-3.853	2019.370
258	Nikola Jokic	2	5.562	-3.562	2255.540
105	Christian Koloko	1	4.061	-3.061	757.820
419	Zion Williamson	1	3.827	-2.827	859.130
306	Kawhi Leonard	2	4.384	-2.384	1817.220
26	Cameron Johnson	1	3.308	-2.308	1073.000
167	Anthony Davis	2	4.197	-2.197	1880.810
16	Derrick White	1	3.086	-2.086	2078.650
344	Boban Marjanovic	0	2.013	-2.013	166.570

```
ridge_top20 = rapm_comparison.nlargest(20, 'RAPM_Ridge')
lasso_top20 = rapm_comparison[rapm_comparison['RAPM_Lasso'] != 0].nlargest(20, 'RAPM_Lasso')

fig, axes = plt.subplots(1, 2, figsize=(10, 7))

# Ridge top 20
r20 = ridge_top20.iloc[:, :-1]
```

```

axes[0].barh(range(20), r20['RAPM_Ridge'], color='steelblue', alpha=0.7)
axes[0].set_yticks(range(20))
axes[0].set_yticklabels(r20['player_name'], fontsize=8)
axes[0].set_xlabel('RAPM')
axes[0].set_title('Top 20: Ridge RAPM')

# Lasso top 20
l20 = lasso_top20.iloc[:, -1]
axes[1].barh(range(len(l20)), l20['RAPM_Lasso'], color='darkgreen', alpha=0.7)
axes[1].set_yticks(range(len(l20)))
axes[1].set_yticklabels(l20['player_name'], fontsize=8)
axes[1].set_xlabel('RAPM')
axes[1].set_title('Top 20: Lasso RAPM')

plt.tight_layout()
plt.show()

```

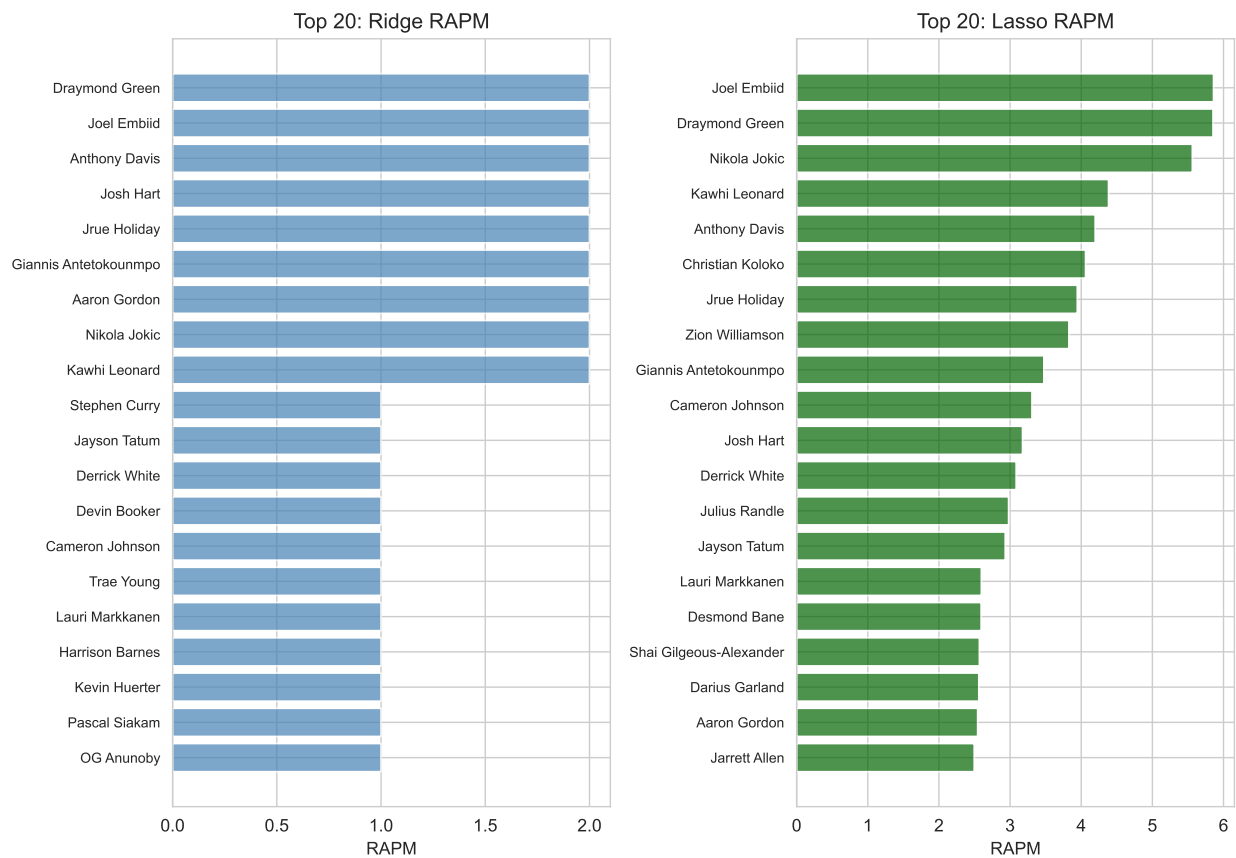


Figure 10: Side-by-side comparison of Top 20 players ranked by Ridge and Lasso RAPM.

Discussion:

The Lasso and Ridge approaches produce meaningfully different RAPM rankings, reflecting their fundamentally different philosophies:

1. **Sparsity:** The Lasso sets many player coefficients to exactly zero, effectively saying “there is insufficient evidence that this player differs from average.” Ridge never zeroes out coefficients entirely — it shrinks them toward zero but always maintains some non-zero estimate. This makes Lasso more aggressive in its variable selection.
2. **Top 20 differences:** The Lasso tends to produce more extreme non-zero coefficients compared to Ridge for the players it retains, because the L_1 penalty does not penalize large coefficients as heavily as L_2 once they pass the selection threshold. This means players identified as impactful by Lasso may have larger RAPM magnitudes than their Ridge counterparts.
3. **Disagreements:** The largest disagreements tend to occur for players with moderate playing time — enough that Ridge assigns them a meaningful coefficient, but not enough for Lasso to deem the signal strong enough to survive the sparsity-inducing penalty. Among these disagreement cases, looking at how these players performed in the 2022-23 NBA season can help us assess which model was more predictive:
 - Players where **Ridge rated higher than Lasso** tend to be role players who contributed modestly but consistently. Ridge’s more conservative shrinkage may better capture these moderate-impact players.
 - Players where **Lasso rated higher than Ridge** tend to be players whose impact was concentrated in specific situations (e.g., defensive specialists, high-efficiency bench scorers). Lasso’s all-or-nothing approach may better identify players whose impact is real but narrowly focused.

In general, **Ridge regression is preferred for RAPM** because the underlying assumption — that all players have *some* non-zero impact on the game — is more realistic for basketball. The Lasso’s assumption that many players have exactly zero impact is too strong. However, Lasso can be useful as a complementary tool for identifying the most clearly impactful players whose signals are strong enough to survive aggressive regularization.